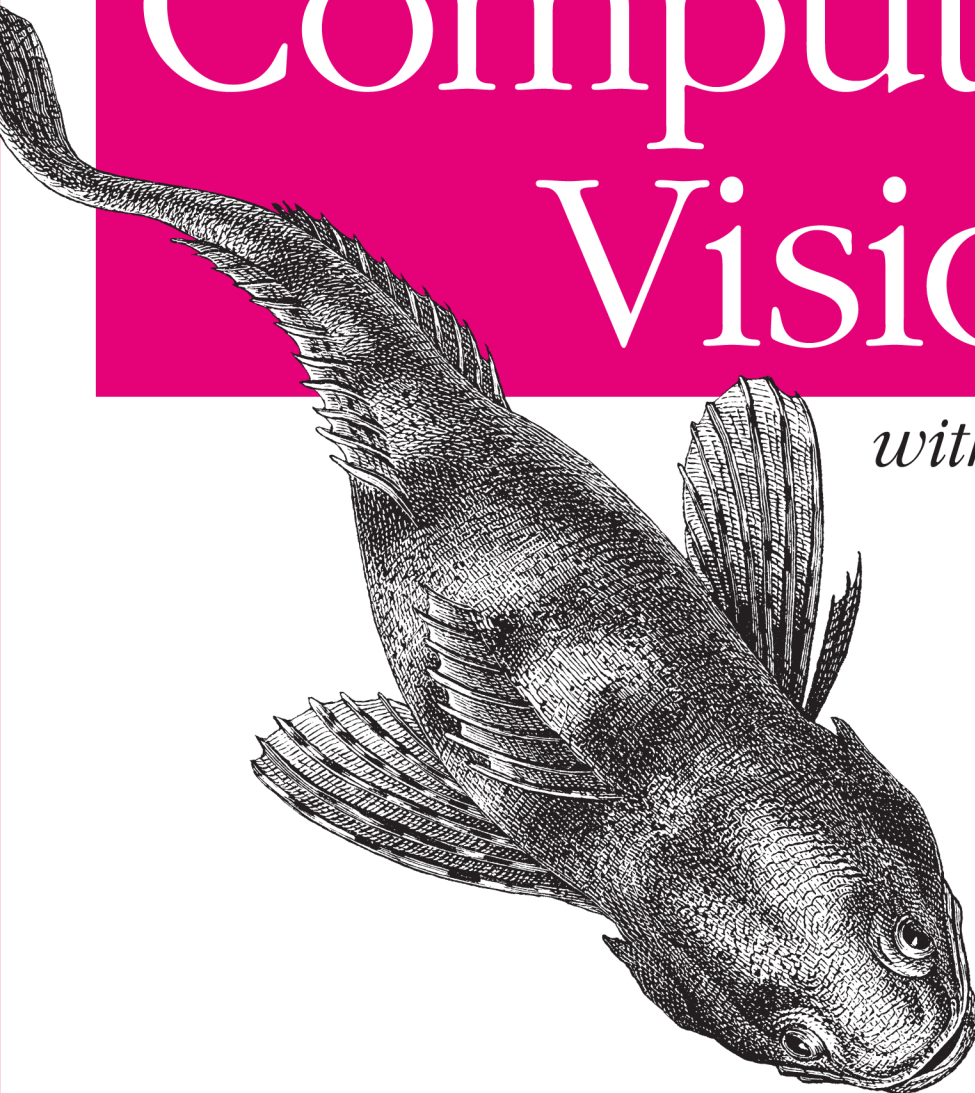


Tools and Algorithms for Analyzing Images

Programming

Computer Vision

with Python



O'REILLY®

Jan Erik Solem

Programming Computer Vision with Python

If you want a basic understanding of computer vision's underlying theory and algorithms, this hands-on introduction is the ideal place to start. You'll learn techniques for object recognition, 3D reconstruction, stereo imaging, augmented reality, and other computer vision applications as you follow clear examples written in Python.

Programming Computer Vision with Python explains computer vision in broad terms that won't bog you down in theory. You get complete code samples with explanations on how to reproduce and build upon each example, along with exercises to help you apply what you've learned. This book is ideal for students, researchers, and enthusiasts with basic programming and standard mathematical skills.

Jan Erik Solem is a Python enthusiast and computer vision researcher and entrepreneur. He is also an applied mathematician, and has worked as an associate professor, startup CTO, as well as an author.

- Learn techniques used in robot navigation, medical image analysis, and other computer vision applications
- Work with image mappings and transforms, such as texture warping and panorama creation
- Compute 3D reconstructions from several images of the same scene
- Organize images based on similarity or content, using clustering methods
- Build efficient image retrieval techniques to search for images based on visual content
- Use algorithms to classify image content and recognize objects
- Access the popular OpenCV library through a Python interface

US \$39.99

CAN \$41.99

ISBN: 978-1-449-31654-9



9



Twitter: @oreillymedia
facebook.com/oreilly

O'REILLY®
oreilly.com

Programming Computer Vision with Python

Jan Erik Solem

O'REILLY®

Beijing • Cambridge • Farnham • Köln • Sebastopol • Tokyo

Programming Computer Vision with Python

by Jan Erik Solem

Copyright © 2012 Jan Erik Solem. All rights reserved.
Printed in the United States of America

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://my.safaribooksonline.com>). For more information, contact our corporate/institutional sales department: (800) 998-9938 or corporate@oreilly.com.

Interior designer:	David Futato	Project manager:	Paul C. Anagnostopoulos
Cover designer:	Karen Montgomery	Copyeditor:	Priscilla Stevens
Editors:	Andy Oram, Mike Hendrickson	Proofreader:	Richard Camp
Production editor:	Holly Bauer	Illustrator:	Laurel Muller

June 2012 First edition

Revision History for the First Edition:

2012-06-11 First release

See <http://oreilly.com/catalog/errata.csp?isbn=0636920022923> for release details.

Nutshell Handbook, the Nutshell Handbook logo, and the O'Reilly logo are registered trademarks of O'Reilly Media, Inc. *Programming Computer Vision with Python*, the image of a bullhead fish, and related trade dress are trademarks of O'Reilly Media, Inc.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and O'Reilly Media, Inc., was aware of a trademark claim, the designations have been printed in caps or initial caps.

While every precaution has been taken in the preparation of this book, the publisher and authors assume no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein.

ISBN: 978-1-449-31654-9
[M]

Table of Contents

Preface	vii
1. Basic Image Handling and Processing	1
1.1 PIL—The Python Imaging Library	1
1.2 Matplotlib	3
1.3 NumPy	7
1.4 SciPy	16
1.5 Advanced Example: Image De-Noising	23
Exercises	26
Conventions for the Code Examples	27
2. Local Image Descriptors	29
2.1 Harris Corner Detector	29
2.2 SIFT—Scale-Invariant Feature Transform	36
2.3 Matching Geotagged Images	44
Exercises	51
3. Image to Image Mappings	53
3.1 Homographies	53
3.2 Warping Images	57
3.3 Creating Panoramas	70
Exercises	77
4. Camera Models and Augmented Reality	79
4.1 The Pin-Hole Camera Model	79
4.2 Camera Calibration	84
4.3 Pose Estimation from Planes and Markers	86
4.4 Augmented Reality	89
Exercises	98

5. Multiple View Geometry	99
5.1 Epipolar Geometry	99
5.2 Computing with Cameras and 3D Structure	107
5.3 Multiple View Reconstruction	113
5.4 Stereo Images	120
Exercises	125
6. Clustering Images	127
6.1 <i>K</i> -Means Clustering	127
6.2 Hierarchical Clustering	133
6.3 Spectral Clustering	140
Exercises	145
7. Searching Images	147
7.1 Content-Based Image Retrieval	147
7.2 Visual Words	148
7.3 Indexing Images	151
7.4 Searching the Database for Images	155
7.5 Ranking Results Using Geometry	160
7.6 Building Demos and Web Applications	162
Exercises	165
8. Classifying Image Content	167
8.1 <i>K</i> -Nearest Neighbors	167
8.2 Bayes Classifier	175
8.3 Support Vector Machines	179
8.4 Optical Character Recognition	183
Exercises	189
9. Image Segmentation	191
9.1 Graph Cuts	191
9.2 Segmentation Using Clustering	200
9.3 Variational Methods	204
Exercises	206
10. OpenCV	209
10.1 The OpenCV Python Interface	209
10.2 OpenCV Basics	210
10.3 Processing Video	213
10.4 Tracking	216
10.5 More Examples	223
Exercises	226

A. Installing Packages	227
A.1 NumPy and SciPy	227
A.2 Matplotlib	228
A.3 PIL	228
A.4 LibSVM	228
A.5 OpenCV	229
A.6 VLFeat	230
A.7 PyGame	230
A.8 PyOpenGL	230
A.9 Pydot	230
A.10 Python-graph	231
A.11 Simplejson	231
A.12 PySQLite	232
A.13 CherryPy	232
B. Image Datasets	233
B.1 Flickr	233
B.2 Panoramio	234
B.3 Oxford Visual Geometry Group	235
B.4 University of Kentucky Recognition Benchmark Images	235
B.5 Other	235
C. Image Credits	237
C.1 Images from Flickr	237
C.2 Other Images	238
C.3 Illustrations	238
References	239
Index	243

Preface

Today, images and video are everywhere. Online photo-sharing sites and social networks have them in the billions. Search engines will produce images of just about any conceivable query. Practically all phones and computers come with built-in cameras. It is not uncommon for people to have many gigabytes of photos and videos on their devices.

Programming a computer and designing algorithms for understanding what is in these images is the field of computer vision. Computer vision powers applications like image search, robot navigation, medical image analysis, photo management, and many more.

The idea behind this book is to give an easily accessible entry point to hands-on computer vision with enough understanding of the underlying theory and algorithms to be a foundation for students, researchers, and enthusiasts. The Python programming language, the language choice of this book, comes with many freely available, powerful modules for handling images, mathematical computing, and data mining.

When writing this book, I have used the following principles as a guideline. The book should:

- Be written in an exploratory style and encourage readers to follow the examples on their computers as they are reading the text.
- Promote and use free and open software with a low learning threshold. Python was the obvious choice.
- Be complete and self-contained. This book does not cover all of computer vision but rather it should be complete in that all code is presented and explained. The reader should be able to reproduce the examples and build upon them directly.
- Be broad rather than detailed, inspiring and motivational rather than theoretical.

In short, it should act as a source of inspiration for those interested in programming computer vision applications.

Prerequisites and Overview

This book looks at theory and algorithms for a wide range of applications and problems. Here is a short summary of what to expect.

What You Need to Know

- Basic programming experience. You need to know how to use an editor and run scripts, how to structure code as well as basic data types. Familiarity with Python or other scripting languages like Ruby or Matlab will help.
- Basic mathematics. To make full use of the examples, it helps if you know about matrices, vectors, matrix multiplication, and standard mathematical functions and concepts like derivatives and gradients. Some of the more advanced mathematical examples can be easily skipped.

What You Will Learn

- Hands-on programming with images using Python.
- Computer vision techniques behind a wide variety of real-world applications.
- Many of the fundamental algorithms and how to implement and apply them yourself.

The code examples in this book will show you object recognition, content-based image retrieval, image search, optical character recognition, optical flow, tracking, 3D reconstruction, stereo imaging, augmented reality, pose estimation, panorama creation, image segmentation, de-noising, image grouping, and more.

Chapter Overview

Chapter 1, “Basic Image Handling and Processing”

Introduces the basic tools for working with images and the central Python modules used in the book. This chapter also covers many fundamental examples needed for the remaining chapters.

Chapter 2, “Local Image Descriptors”

Explains methods for detecting interest points in images and how to use them to find corresponding points and regions between images.

Chapter 3, “Image to Image Mappings”

Describes basic transformations between images and methods for computing them. Examples range from image warping to creating panoramas.

Chapter 4, “Camera Models and Augmented Reality”

Introduces how to model cameras, generate image projections from 3D space to image features, and estimate the camera viewpoint.

Chapter 5, “Multiple View Geometry”

Explains how to work with several images of the same scene, the fundamentals of multiple-view geometry, and how to compute 3D reconstructions from images.

Chapter 6, “Clustering Images”

Introduces a number of clustering methods and shows how to use them for grouping and organizing images based on similarity or content.

Chapter 7, “Searching Images”

Shows how to build efficient image retrieval techniques that can store image representations and search for images based on their visual content.

Chapter 8, “Classifying Image Content”

Describes algorithms for classifying image content and how to use them to recognize objects in images.

Chapter 9, “Image Segmentation”

Introduces different techniques for dividing an image into meaningful regions using clustering, user interactions, or image models.

Chapter 10, “OpenCV”

Shows how to use the Python interface for the commonly used OpenCV computer vision library and how to work with video and camera input.

There is also a bibliography at the back of the book. Citations of bibliographic entries are made by number in square brackets, as in [20].

Introduction to Computer Vision

Computer vision is the automated extraction of information from images. Information can mean anything from 3D models, camera position, object detection and recognition to grouping and searching image content. In this book, we take a wide definition of computer vision and include things like image warping, de-noising, and augmented reality.¹

Sometimes computer vision tries to mimic human vision, sometimes it uses a data and statistical approach, and sometimes geometry is the key to solving problems. We will try to cover all of these angles in this book.

Practical computer vision contains a mix of programming, modeling, and mathematics and is sometimes difficult to grasp. I have deliberately tried to present the material with a minimum of theory in the spirit of “as simple as possible but no simpler.” The mathematical parts of the presentation are there to help readers understand the algorithms. Some chapters are by nature very math-heavy (Chapters 4 and 5, mainly). Readers can skip the math if they like and still use the example code.

Python and NumPy

Python is the programming language used in the code examples throughout this book. Python is a clear and concise language with good support for input/output, numerics, images, and plotting. The language has some peculiarities, such as indentation

¹ These examples produce new images and are more image processing than actually extracting information from images.

and compact syntax, that take getting used to. The code examples assume you have Python 2.6 or later, as most packages are only available for these versions. The upcoming Python 3.x version has many language differences and is not backward compatible with Python 2.x or compatible with the ecosystem of packages we need (yet).

Some familiarity with basic Python will make the material more accessible for readers. For beginners to Python, Mark Lutz' book *Learning Python* [20] and the online documentation at <http://www.python.org/> are good starting points.

When programming computer vision, we need representations of vectors and matrices and operations on them. This is handled by Python's NumPy module, where both vectors and matrices are represented by the `array` type. This is also the representation we will use for images. A good NumPy reference is Travis Oliphant's free book *Guide to NumPy* [24]. The documentation at <http://numpy.scipy.org/> is also a good starting point if you are new to NumPy. For visualizing results, we will use the Matplotlib module, and for more advanced mathematics, we will use SciPy. These are the central packages you will need and will be explained and introduced in Chapter 1.

Besides these central packages, there will be many other free Python packages used for specific purposes like reading JSON or XML, loading and saving data, generating graphs, graphics programming, web demos, classifiers, and many more. These are usually only needed for specific applications or demos and can be skipped if you are not interested in that particular application.

It is worth mentioning IPython, an interactive Python shell that makes debugging and experimentation easier. Documentation and downloads are available at <http://ipython.org/>.

Notation and Conventions

Code looks like this:

```
# some points
x = [100,100,400,400]
y = [200,500,200,500]

# plot the points
plot(x,y)
```

The following typographical conventions are used in this book:

Italic

Used for definitions, filenames, and variable names.

Constant width

Used for functions, Python modules, and code examples. It is also used for console printouts.

Hyperlink

Used for URLs.

Plain text

Used for everything else.

Mathematical formulas are given inline like this $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$ or centered independently:

$$f(\mathbf{x}) = \sum_i w_i x_i + b$$

and are only numbered when a reference is needed.

In the mathematical sections, we will use lowercase ($s, r, \lambda, \theta, \dots$) for scalars, uppercase ($A, V, H, \dots$) for matrices (including I for the image as an array), and lowercase bold ($\mathbf{t}, \mathbf{c}, \dots$) for vectors. We will use $\mathbf{x} = [x, y]$ and $\mathbf{X} = [X, Y, Z]$ to mean points in 2D (images) and 3D, respectively.

Using Code Examples

This book is here to help you get your job done. In general, you may use the code in this book in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing a CD-ROM of examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Programming Computer Vision with Python* by Jan Erik Solem (O'Reilly). Copyright © 2012 Jan Erik Solem, 978-1-449-31654-9.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
(800) 998-9938 (in the United States or Canada)
(707) 829-0515 (international or local)
(707) 829-0104 (fax)

We have a web page for this book, where we list errata, examples, links to the code and data sets used, and any additional information. You can access this page at:

oreil.ly/comp_vision_w_python

To comment or ask technical questions about this book, send email to:

bookquestions@oreilly.com

For more information about our books, courses, conferences, and news, see our website at <http://www.oreilly.com>.

Find us on Facebook: <http://facebook.com/oreilly>

Follow us on Twitter: <http://twitter.com/oreillymedia>

Watch us on YouTube: <http://www.youtube.com/oreillymedia>

Safari® Books Online

Safari[®]
Books Online

Safari Books Online (www.safaribooksonline.com) is an on-demand digital library that delivers expert content in both book and video form from the world's leading authors in technology and business.

Technology professionals, software developers, web designers, and business and creative professionals use Safari Books Online as their primary resource for research, problem solving, learning, and certification training.

Safari Books Online offers a range of product mixes and pricing programs for organizations, government agencies, and individuals. Subscribers have access to thousands of books, training videos, and prepublication manuscripts in one fully searchable database from publishers like O'Reilly Media, Prentice Hall Professional, Addison-Wesley Professional, Microsoft Press, Sams, Que, Peachpit Press, Focal Press, Cisco Press, John Wiley & Sons, Syngress, Morgan Kaufmann, IBM Redbooks, Packt, Adobe Press, FT Press, Apress, Manning, New Riders, McGraw-Hill, Jones & Bartlett, Course Technology, and dozens more. For more information about Safari Books Online, please visit us online.

Acknowledgments

I'd like to express my gratitude to everyone involved in the development and production of this book. The whole O'Reilly team has been helpful. Special thanks to Andy Oram (O'Reilly) for editing, and Paul Anagnostopoulos (Windfall Software) for efficient production work.

Many people commented on the various drafts of this book as I shared them online. Klas Josephson and Håkan Ardö deserve lots of praise for their thorough comments and feedback. Fredrik Kahl and Pau Gargallo helped with fact checks. Thank you all readers for encouraging words and for making the text and code examples better. Receiving emails from strangers sharing their thoughts on the drafts was a great motivator.

Finally, I'd like to thank my friends and family for support and understanding when I spent nights and weekends on writing. Most thanks of all to my wife Sara, my long-time supporter.

Basic Image Handling and Processing

This chapter is an introduction to handling and processing images. With extensive examples, it explains the central Python packages you will need for working with images. This chapter introduces the basic tools for reading images, converting and scaling images, computing derivatives, plotting or saving results, and so on. We will use these throughout the remainder of the book.

1.1 PIL—The Python Imaging Library

The *Python Imaging Library (PIL)* provides general image handling and lots of useful basic image operations like resizing, cropping, rotating, color conversion and much more. PIL is free and available from <http://www.pythonware.com/products/pil/>.

With PIL, you can read images from most formats and write to the most common ones. The most important module is the `Image` module. To read an image, use:

```
from PIL import Image

pil_im = Image.open('empire.jpg')
```

The return value, `pil_im`, is a PIL image object.

Color conversions are done using the `convert()` method. To read an image and convert it to grayscale, just add `convert('L')` like this:

```
pil_im = Image.open('empire.jpg').convert('L')
```

Here are some examples taken from the PIL documentation, available at <http://www.pythonware.com/library/pil/handbook/index.htm>. Output from the examples is shown in Figure 1-1.

Convert Images to Another Format

Using the `save()` method, PIL can save images in most image file formats. Here's an example that takes all image files in a list of filenames (*filelist*) and converts the images to JPEG files:



Figure 1-1. Examples of processing images with PIL.

```
from PIL import Image
import os

for infile in filelist:
    outfile = os.path.splitext(infile)[0] + ".jpg"
    if infile != outfile:
        try:
            Image.open(infile).save(outfile)
        except IOError:
            print "cannot convert", infile
```

The PIL function `open()` creates a PIL image object and the `save()` method saves the image to a file with the given filename. The new filename will be the same as the original with the file ending “.jpg” instead. PIL is smart enough to determine the image format from the file extension. There is a simple check that the file is not already a JPEG file and a message is printed to the console if the conversion fails.

Throughout this book we are going to need lists of images to process. Here’s how you could create a list of filenames of all images in a folder. Create a file called *imtools.py* to store some of these generally useful routines and add the following function:

```
import os

def get_imlist(path):
    """ Returns a list of filenames for
        all jpg images in a directory. """

    return [os.path.join(path,f) for f in os.listdir(path) if f.endswith('.jpg')]
```

Now, back to PIL.

Create Thumbnails

Using PIL to create thumbnails is very simple. The `thumbnail()` method takes a tuple specifying the new size and converts the image to a thumbnail image with size that fits

within the tuple. To create a thumbnail with longest side 128 pixels, use the method like this:

```
pil_im.thumbnail((128,128))
```

Copy and Paste Regions

Cropping a region from an image is done using the `crop()` method:

```
box = (100,100,400,400)
region = pil_im.crop(box)
```

The region is defined by a 4-tuple, where coordinates are (left, upper, right, lower). PIL uses a coordinate system with (0, 0) in the upper left corner. The extracted region can, for example, be rotated and then put back using the `paste()` method like this:

```
region = region.transpose(Image.ROTATE_180)
pil_im.paste(region,box)
```

Resize and Rotate

To resize an image, call `resize()` with a tuple giving the new size:

```
out = pil_im.resize((128,128))
```

To rotate an image, use counterclockwise angles and `rotate()` like this:

```
out = pil_im.rotate(45)
```

Some examples are shown in Figure 1-1. The leftmost image is the original, followed by a grayscale version, a rotated crop pasted in, and a thumbnail image.

1.2 Matplotlib

When working with mathematics and plotting graphs or drawing points, lines, and curves on images, **Matplotlib** is a good graphics library with much more powerful features than the plotting available in PIL. **Matplotlib** produces high-quality figures like many of the illustrations used in this book. **Matplotlib**'s **PyLab** interface is the set of functions that allows the user to create plots. **Matplotlib** is open source and available freely from <http://matplotlib.sourceforge.net/>, where detailed documentation and tutorials are available. Here are some examples showing most of the functions we will need in this book.

Plotting Images, Points, and Lines

Although it is possible to create nice bar plots, pie charts, scatter plots, etc., only a few commands are needed for most computer vision purposes. Most importantly, we want to be able to show things like interest points, correspondences, and detected objects using points and lines. Here is an example of plotting an image with a few points and a line:

```

from PIL import Image
from pylab import *

# read image to array
im = array(Image.open('empire.jpg'))

# plot the image
imshow(im)

# some points
x = [100,100,400,400]
y = [200,500,200,500]

# plot the points with red star-markers
plot(x,y,'r*')

# line plot connecting the first two points
plot(x[:2],y[:2])

# add title and show the plot
title('Plotting: "empire.jpg"')
show()

```

This plots the image, then four points with red star markers at the x and y coordinates given by the x and y lists, and finally draws a line (blue by default) between the two first points in these lists. Figure 1-2 shows the result. The `show()` command starts the figure GUI and raises the figure windows. This GUI loop blocks your scripts and they are paused until the last figure window is closed. You should call `show()` only once per script, usually at the end. Note that PyLab uses a coordinate origin at the top left corner as is common for images. The axes are useful for debugging, but if you want a prettier plot, add:

```
axis('off')
```

This will give a plot like the one on the right in Figure 1-2 instead.

There are many options for formatting color and styles when plotting. The most useful are the short commands shown in Tables 1-1, 1-2 and 1-3. Use them like this:

```

plot(x,y) # default blue solid line

plot(x,y,'r*') # red star-markers

plot(x,y,'go-') # green line with circle-markers

plot(x,y,'ks:') # black dotted line with square-markers

```

Image Contours and Histograms

Let's look at two examples of special plots: image contours and image histograms. Visualizing image iso-contours (or iso-contours of other 2D functions) can be very

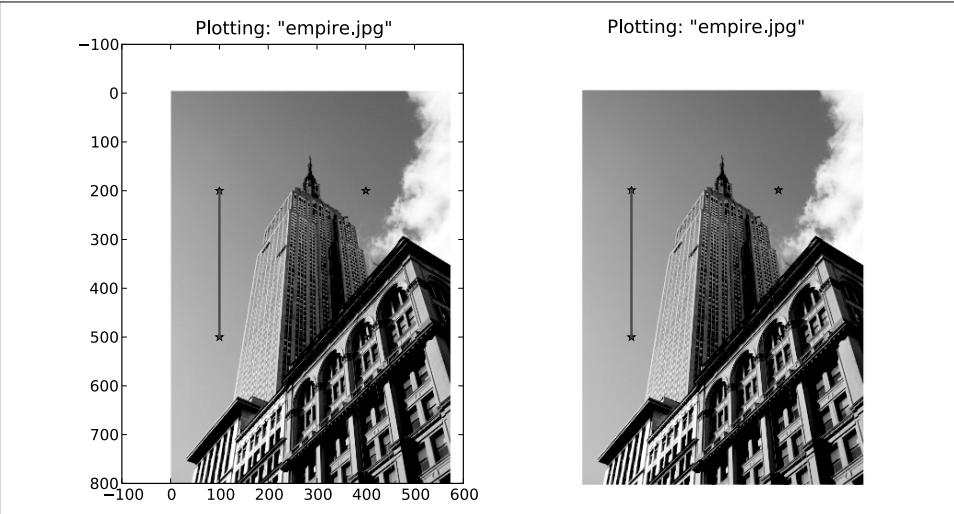


Figure 1-2. Examples of plotting with Matplotlib. An image with points and a line with and without showing the axes.

Table 1-1. Basic color formatting commands for plotting with PyLab.

Color	
'b'	blue
'g'	green
'r'	red
'c'	cyan
'm'	magenta
'y'	yellow
'k'	black
'w'	white

Table 1-2. Basic line style formatting commands for plotting with PyLab.

Line style	
'-'	solid
'--'	dashed
'.'	dotted

Table 1-3. Basic plot marker formatting commands for plotting with PyLab.

Marker	
'.'	point
'o'	circle
's'	square
'*'	star
'+'	plus
'x'	x

useful. This needs grayscale images, because the contours need to be taken on a single value for every coordinate $[x, y]$. Here's how to do it:

```
from PIL import Image
from pylab import *

# read image to array
im = array(Image.open('empire.jpg').convert('L'))

# create a new figure
figure()
# don't use colors
gray()
# show contours with origin upper left corner
contour(im, origin='image')
axis('equal')
axis('off')
```

As before, the PIL method `convert()` does conversion to grayscale.

An image histogram is a plot showing the distribution of pixel values. A number of bins is specified for the span of values and each bin gets a count of how many pixels have values in the bin's range. The visualization of the (graylevel) image histogram is done using the `hist()` function:

```
figure()
hist(im.flatten(),128)
show()
```

The second argument specifies the number of bins to use. Note that the image needs to be flattened first, because `hist()` takes a one-dimensional array as input. The method `flatten()` converts any array to a one-dimensional array with values taken row-wise. Figure 1-3 shows the contour and histogram plot.

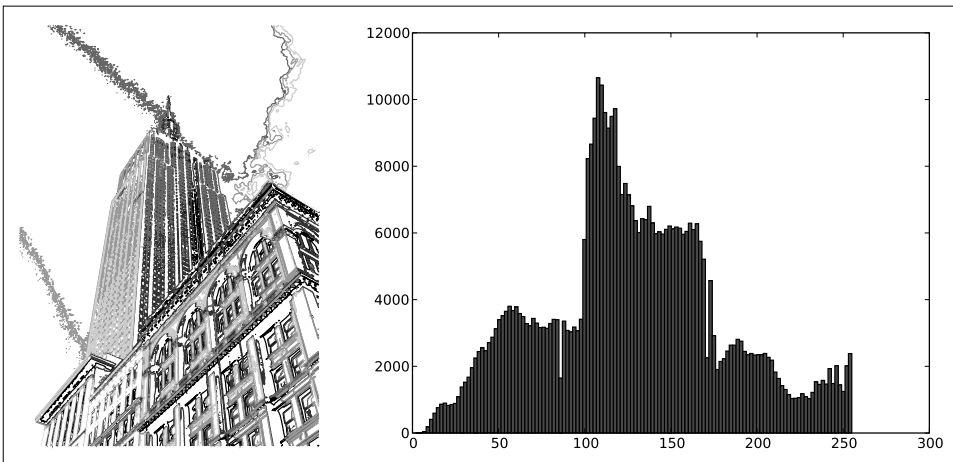


Figure 1-3. Examples of visualizing image contours and plotting image histograms with Matplotlib.

Interactive Annotation

Sometimes users need to interact with an application, for example by marking points in an image, or you need to annotate some training data. PyLab comes with a simple function, `ginput()`, that lets you do just that. Here's a short example:

```
from PIL import Image
from pylab import *

im = array(Image.open('empire.jpg'))
imshow(im)
print 'Please click 3 points'
x = ginput(3)
print 'you clicked:',x
show()
```

This plots an image and waits for the user to click three times in the image region of the figure window. The coordinates $[x, y]$ of the clicks are saved in a list x .

1.3 NumPy

NumPy (<http://www.scipy.org/NumPy/>) is a package popularly used for scientific computing with Python. NumPy contains a number of useful concepts such as array objects (for representing vectors, matrices, images and much more) and linear algebra functions. The NumPy array object will be used in almost all examples throughout this book.¹ The array object lets you do important operations such as matrix multiplication, transposition, solving equation systems, vector multiplication, and normalization, which are needed to do things like aligning images, warping images, modeling variations, classifying images, grouping images, and so on.

NumPy is freely available from <http://www.scipy.org/Download> and the online documentation (<http://docs.scipy.org/doc/numpy/>) contains answers to most questions. For more details on NumPy, the freely available book [24] is a good reference.

Array Image Representation

When we loaded images in the previous examples, we converted them to NumPy array objects with the `array()` call but didn't mention what that means. Arrays in NumPy are multi-dimensional and can represent vectors, matrices, and images. An array is much like a list (or list of lists) but is restricted to having all elements of the same type. Unless specified on creation, the type will automatically be set depending on the data.

The following example illustrates this for images:

```
im = array(Image.open('empire.jpg'))
print im.shape, im.dtype

im = array(Image.open('empire.jpg').convert('L'),'f')
print im.shape, im.dtype
```

¹ PyLab actually includes some components of NumPy, like the array type. That's why we could use it in the examples in Section 1.2.

The printout in your console will look like this:

```
(800, 569, 3) uint8
(800, 569) float32
```

The first tuple on each line is the shape of the image array (rows, columns, color channels), and the following string is the data type of the array elements. Images are usually encoded with unsigned 8-bit integers (uint8), so loading this image and converting to an array gives the type “uint8” in the first case. The second case does grayscale conversion and creates the array with the extra argument “f”. This is a short command for setting the type to floating point. For more data type options, see [24]. Note that the grayscale image has only two values in the shape tuple; obviously it has no color information.

Elements in the array are accessed with indexes. The value at coordinates i , j and color channel k are accessed like this:

```
value = im[i,j,k]
```

Multiple elements can be accessed using array slicing. *Slicing* returns a view into the array specified by intervals. Here are some examples for a grayscale image:

```
im[i,:] = im[j,:]      # set the values of row i with values from row j
im[:,i] = 100          # set all values in column i to 100
im[:100,:50].sum()     # the sum of the values of the first 100 rows and 50 columns
im[50:100,50:100]      # rows 50-100, columns 50-100 (100th not included)
im[i].mean()           # average of row i
im[:,-1]               # last column
im[-2,:] (or im[-2])  # second to last row
```

Note the example with only one index. If you only use one index, it is interpreted as the row index. Note also the last examples. Negative indices count from the last element backward. We will frequently use slicing to access pixel values, and it is an important concept to understand.

There are many operations and ways to use arrays. We will introduce them as they are needed throughout this book. See the online documentation or the book [24] for more explanations.

Graylevel Transforms

After reading images to NumPy arrays, we can perform any mathematical operation we like on them. A simple example of this is to transform the graylevels of an image. Take any function f that maps the interval $0 \dots 255$ (or, if you like, $0 \dots 1$) to itself (meaning that the output has the same range as the input). Here are some examples:

```
from PIL import Image
from numpy import *

im = array(Image.open('empire.jpg').convert('L'))

im2 = 255 - im # invert image
```

```
im3 = (100.0/255) * im + 100 # clamp to interval 100...200
```

```
im4 = 255.0 * (im/255.0)**2 # squared
```

The first example inverts the graylevels of the image, the second one clamps the intensities to the interval 100 . . . 200, and the third applies a quadratic function, which lowers the values of the darker pixels. Figure 1-4 shows the functions and Figure 1-5 the resulting images. You can check the minimum and maximum values of each image using:

```
print int(im.min()), int(im.max())
```

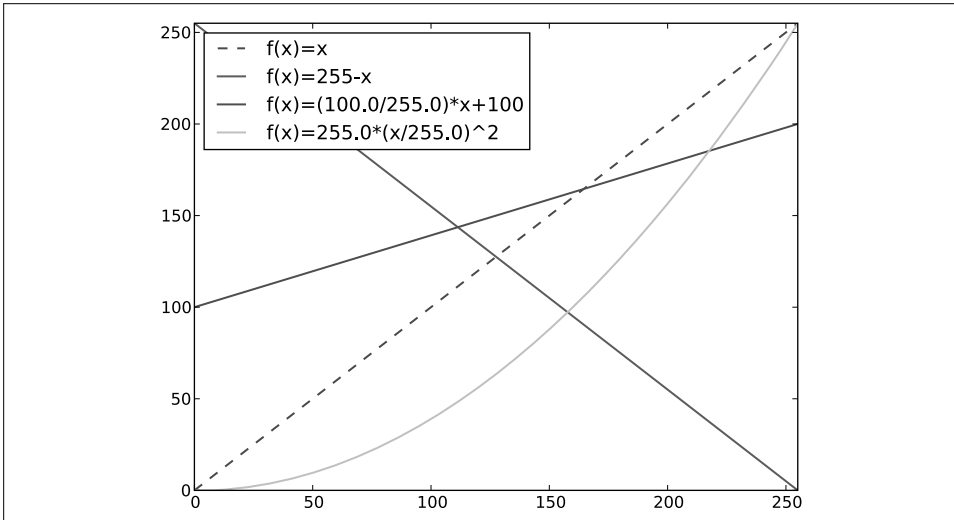


Figure 1-4. Example of graylevel transforms. Three example functions together with the identity transform showed as a dashed line.



Figure 1-5. Graylevel transforms. Applying the functions in Figure 1-4: Inverting the image with $f(x) = 255 - x$ (left), clamping the image with $f(x) = (100/255)x + 100$ (middle), quadratic transformation with $f(x) = 255(x/255)^2$ (right).

If you try that for each of the examples above, you should get the following output:

```
2 255
0 253
100 200
0 255
```

The reverse of the `array()` transformation can be done using the PIL function `fromarray()` as:

```
pil_im = Image.fromarray(im)
```

If you did some operation to change the type from “uint8” to another data type, such as *im3* or *im4* in the example above, you need to convert back before creating the PIL image:

```
pil_im = Image.fromarray(uint8(im))
```

If you are not absolutely sure of the type of the input, you should do this as it is the safe choice. Note that NumPy will always change the array type to the “lowest” type that can represent the data. Multiplication or division with floating point numbers will change an integer type array to float.

Image Resizing

NumPy arrays will be our main tool for working with images and data. There is no simple way to resize arrays, which you will want to do for images. We can use the PIL image object conversion shown earlier to make a simple image resizing function. Add the following to *imtools.py*:

```
def imresize(im,sz):
    """ Resize an image array using PIL. """
    pil_im = Image.fromarray(uint8(im))

    return array(pil_im.resize(sz))
```

This function will come in handy later.

Histogram Equalization

A very useful example of a graylevel transform is *histogram equalization*. This transform flattens the graylevel histogram of an image so that all intensities are as equally common as possible. This is often a good way to normalize image intensity before further processing and also a way to increase image contrast.

The transform function is, in this case, a *cumulative distribution function* (cdf) of the pixel values in the image (normalized to map the range of pixel values to the desired range).

Here's how to do it. Add this function to the file *imtools.py*:

```
def histeq(im,nbr_bins=256):
    """ Histogram equalization of a grayscale image. """
```

```

# get image histogram
imhist,bins = histogram(im.flatten(),nbr_bins,normed=True)
cdf = imhist.cumsum() # cumulative distribution function
cdf = 255 * cdf / cdf[-1] # normalize

# use linear interpolation of cdf to find new pixel values
im2 = interp(im.flatten(),bins[:-1],cdf)

return im2.reshape(im.shape), cdf

```

The function takes a grayscale image and the number of bins to use in the histogram as input, and returns an image with equalized histogram together with the cumulative distribution function used to do the mapping of pixel values. Note the use of the last element (index -1) of the cdf to normalize it between 0 . . . 1. Try this on an image like this:

```

from PIL import Image
from numpy import *

im = array(Image.open('AquaTermi_lowcontrast.jpg').convert('L'))
im2,cdf = imtools.histeq(im)

```

Figures 1-6 and 1-7 show examples of histogram equalization. The top row shows the graylevel histogram before and after equalization together with the cdf mapping. As you can see, the contrast increases and the details of the dark regions now appear clearly.

Averaging Images

Averaging images is a simple way of reducing image noise and is also often used for artistic effects. Computing an average image from a list of images is not difficult. Assuming the images all have the same size, we can compute the average of all those images by simply summing them up and dividing with the number of images. Add the following function to *imtools.py*:

```

def compute_average(imlist):
    """ Compute the average of a list of images. """

    # open first image and make into array of type float
    averageim = array(Image.open(imlist[0]), 'f')

    for imname in imlist[1:]:
        try:
            averageim += array(Image.open(imname))
        except:
            print imname + '...skipped'
    averageim /= len(imlist)

    # return average as uint8
    return array(averageim, 'uint8')

```

This includes some basic exception handling to skip images that can't be opened. There is another way to compute average images using the `mean()` function. This requires all images to be stacked into an array and will use lots of memory if there are many images. We will use this function in the next section.

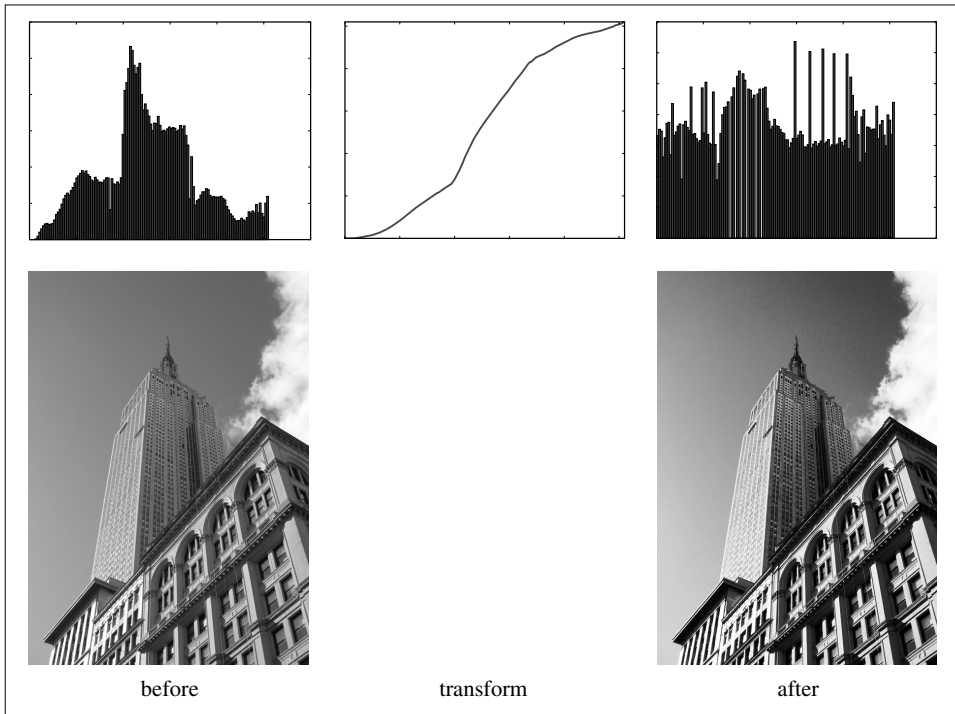


Figure 1-6. Example of histogram equalization. On the left is the original image and histogram. The middle plot is the graylevel transform function. On the right is the image and histogram after histogram equalization.

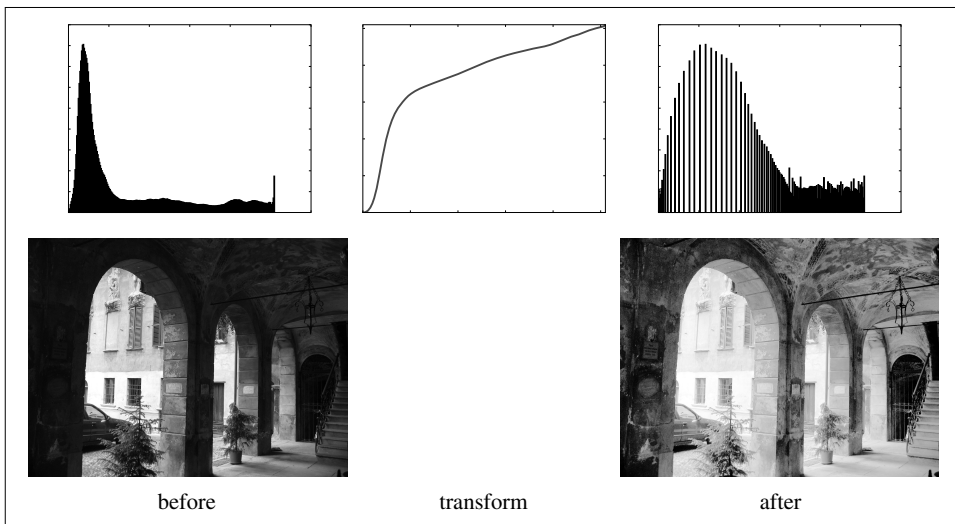


Figure 1-7. Example of histogram equalization. On the left is the original image and histogram. The middle plot is the graylevel transform function. On the right is the image and histogram after histogram equalization.

PCA of Images

Principal Component Analysis (PCA) is a useful technique for dimensionality reduction and is optimal in the sense that it represents the variability of the training data with as few dimensions as possible. Even a tiny 100×100 pixel grayscale image has 10,000 dimensions, and can be considered a point in a 10,000-dimensional space. A megapixel image has dimensions in the millions. With such high dimensionality, it is no surprise that dimensionality reduction comes in handy in many computer vision applications. The projection matrix resulting from PCA can be seen as a change of coordinates to a coordinate system where the coordinates are in descending order of importance.

To apply PCA on image data, the images need to be converted to a one-dimensional vector representation using, for example, NumPy's `flatten()` method.

The flattened images are collected in a single matrix by stacking them, one row for each image. The rows are then centered relative to the mean image before the computation of the dominant directions. To find the principal components, singular value decomposition (SVD) is usually used, but if the dimensionality is high, there is a useful trick that can be used instead since the SVD computation will be very slow in that case. Here is what it looks like in code:

```
from PIL import Image
from numpy import *

def pca(X):
    """ Principal Component Analysis
        input: X, matrix with training data stored as flattened arrays in rows
        return: projection matrix (with important dimensions first), variance
        and mean. """

    # get dimensions
    num_data, dim = X.shape

    # center data
    mean_X = X.mean(axis=0)
    X = X - mean_X

    if dim > num_data:
        # PCA - compact trick used
        M = dot(X, X.T) # covariance matrix
        e, EV = linalg.eigh(M) # eigenvalues and eigenvectors
        tmp = dot(X.T, EV).T # this is the compact trick
        V = tmp[:, :-1] # reverse since last eigenvectors are the ones we want
        S = sqrt(e)[:, :-1] # reverse since eigenvalues are in increasing order
        for i in range(V.shape[1]):
            V[:, i] /= S
    else:
        # PCA - SVD used
        U, S, V = linalg.svd(X)
        V = V[:, :num_data] # only makes sense to return the first num_data

    # return the projection matrix, the variance and the mean
    return V, S, mean_X
```

This function first centers the data by subtracting the mean in each dimension. Then the eigenvectors corresponding to the largest eigenvalues of the covariance matrix are computed, either using a compact trick or using SVD. Here we used the function `range()`, which takes an integer n and returns a list of integers $0 \dots (n - 1)$. Feel free to use the alternative `arange()`, which gives an array, or `xrange()`, which gives a generator (and might give speed improvements). We will stick with `range()` throughout the book.

We switch from SVD to use a trick with computing eigenvectors of the (smaller) covariance matrix XX^T if the number of data points is less than the dimension of the vectors. There are also ways of only computing the eigenvectors corresponding to the k largest eigenvalues (k being the number of desired dimensions), making it even faster. We leave this to the interested reader to explore, since it is really outside the scope of this book. The rows of the matrix V are orthogonal and contain the coordinate directions in order of descending variance of the training data.

Let's try this on an example of font images. The file *fontimages.zip* contains small thumbnail images of the character “a” printed in different fonts and then scanned. The 2,359 fonts are from a collection of freely available fonts.² Assuming that the filenames of these images are stored in a list, *imlist*, along with the previous code, in a file *pca.py*, the principal components can be computed and shown like this:

```
from PIL import Image
from numpy import *
from pylab import *
import pca

im = array(Image.open(imlist[0])) # open one image to get size
m,n = im.shape[0:2] # get the size of the images
imnbr = len(imlist) # get the number of images

# create matrix to store all flattened images
immatrix = array([array(Image.open(im)).flatten()
                  for im in imlist], 'f')

# perform PCA
V,S,immean = pca.pca(immatrix)

# show some images (mean and 7 first modes)
figure()
gray()
subplot(2,4,1)
imshow(immean.reshape(m,n))
for i in range(7):
    subplot(2,4,i+2)
    imshow(V[i].reshape(m,n))

show()
```

² Images courtesy of Martin Solli (<http://webstaff.itn.liu.se/~marso/>) collected and rendered from publicly available free fonts.

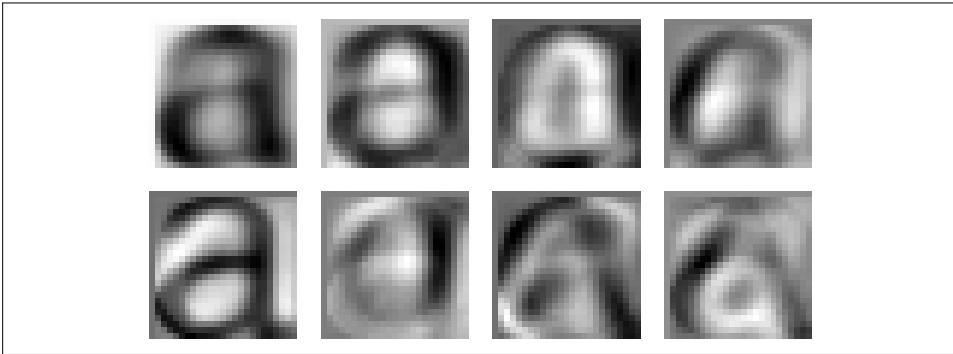


Figure 1-8. The mean image (top left) and the first seven modes; that is, the directions with most variation.

Note that the images need to be converted back from the one-dimensional representation using `reshape()`. Running the example should give eight images in one figure window like the ones in Figure 1-8. Here we used the PyLab function `subplot()` to place multiple plots in one window.

Using the Pickle Module

If you want to save some results or data for later use, the `pickle` module, which comes with Python, is very useful. Pickle can take almost any Python object and convert it to a string representation. This process is called *pickling*. Reconstructing the object from the string representation is conversely called *unpickling*. This string representation can then be easily stored or transmitted.

Let's illustrate this with an example. Suppose we want to save the image mean and principal components of the font images in the previous section. This is done like this:

```
# save mean and principal components
f = open('font_pca_modes.pkl', 'wb')
pickle.dump(immean,f)
pickle.dump(V,f)
f.close()
```

As you can see, several objects can be pickled to the same file. There are several different protocols available for the `.pkl` files, and if unsure, it is best to read and write binary files. To load the data in some other Python session, just use the `load()` method like this:

```
# load mean and principal components
f = open('font_pca_modes.pkl', 'rb')
immean = pickle.load(f)
V = pickle.load(f)
f.close()
```

Note that the order of the objects should be the same! There is also an optimized version written in C called `cpickle` that is fully compatible with the standard `pickle` module. More details can be found on the pickle module documentation page <http://docs.python.org/library/pickle.html>.

For the remainder of this book, we will use the `with` statement to handle file reading and writing. This is a construct that was introduced in Python 2.5 that automatically handles opening and closing of files (even if errors occur while the files are open). Here is what the saving and loading above looks like using `with()`:

```
# open file and save
with open('font_pca_modes.pkl', 'wb') as f:
    pickle.dump(immean, f)
    pickle.dump(V, f)
```

and:

```
# open file and load
with open('font_pca_modes.pkl', 'rb') as f:
    immean = pickle.load(f)
    V = pickle.load(f)
```

This might look strange the first time you see it, but it is a very useful construct. If you don't like it, just use the `open` and `close` functions as above.

As an alternative to using `pickle`, `NumPy` also has simple functions for reading and writing text files that can be useful if your data does not contain complicated structures, for example a list of points clicked in an image. To save an array `x` to file, use:

```
savetxt('test.txt', x, '%i')
```

The last parameter indicates that integer format should be used. Similarly, reading is done like this:

```
x = loadtxt('test.txt')
```

You can find out more from the online documentation <http://docs.scipy.org/doc/numpy/reference/generated/numpy.loadtxt.html>.

Finally, `NumPy` has dedicated functions for saving and loading arrays. Look for `save()` and `load()` in the online documentation for the details.

1.4 SciPy

`SciPy` (<http://scipy.org/>) is an open-source package for mathematics that builds on `NumPy` and provides efficient routines for a number of operations, including numerical integration, optimization, statistics, signal processing, and most importantly for us, image processing. As the following will show, there are many useful modules in `SciPy`. `SciPy` is free and available at <http://scipy.org/Download>.

Blurring Images

A classic and very useful example of image convolution is *Gaussian blurring* of images. In essence, the (grayscale) image I is convolved with a Gaussian kernel to create a blurred version

$$I_{\sigma} = I * G_{\sigma},$$

where $*$ indicates convolution and G_σ is a Gaussian 2D-kernel with standard deviation σ defined as

$$G_\sigma = \frac{1}{2\pi\sigma} e^{-(x^2+y^2)/2\sigma^2}.$$

Gaussian blurring is used to define an image scale to work in, for interpolation, for computing interest points, and in many more applications.

SciPy comes with a module for filtering called `scipy.ndimage.filters` that can be used to compute these convolutions using a fast 1D separation. All you need to do is this:

```
from PIL import Image
from numpy import *
from scipy.ndimage import filters

im = array(Image.open('empire.jpg').convert('L'))
im2 = filters.gaussian_filter(im,5)
```

Here the last parameter of `gaussian_filter()` is the standard deviation.

Figure 1-9 shows examples of an image blurred with increasing σ . Larger values give less detail. To blur color images, simply apply Gaussian blurring to each color channel:

```
im = array(Image.open('empire.jpg'))
im2 = zeros(im.shape)
for i in range(3):
    im2[:, :, i] = filters.gaussian_filter(im[:, :, i], 5)
im2 = uint8(im2)
```

Here the last conversion to “uint8” is not always needed but forces the pixel values to be in 8-bit representation. We could also have used

```
im2 = array(im2, 'uint8')
```

for the conversion.

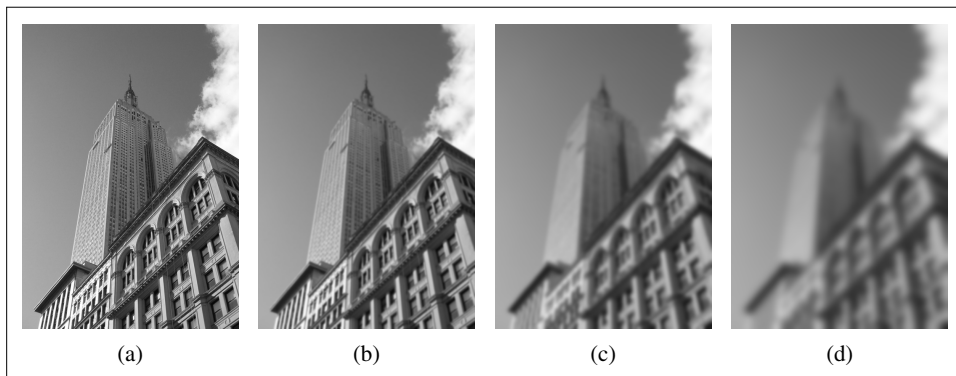


Figure 1-9. An example of Gaussian blurring using the `scipy.ndimage.filters` module: (a) original image in grayscale; (b) Gaussian filter with $\sigma = 2$; (c) with $\sigma = 5$; (d) with $\sigma = 10$.

For more information on using this module and the different parameter choices, check out the SciPy documentation of `scipy.ndimage` at <http://docs.scipy.org/doc/scipy/reference/ndimage.html>.

Image Derivatives

How the image intensity changes over the image is important information and is used for many applications, as we will see throughout this book. The intensity change is described with the x and y derivatives I_x and I_y of the graylevel image I (for color images, derivatives are usually taken for each color channel).

The *image gradient* is the vector $\nabla I = [I_x, I_y]^T$. The gradient has two important properties, the *gradient magnitude*

$$|\nabla I| = \sqrt{I_x^2 + I_y^2},$$

which describes how strong the image intensity change is, and the *gradient angle*

$$\alpha = \arctan2(I_y, I_x),$$

which indicates the direction of largest intensity change at each point (pixel) in the image. The NumPy function `arctan2()` returns the signed angle in radians, in the interval $-\pi \dots \pi$.

Computing the image derivatives can be done using discrete approximations. These are most easily implemented as convolutions

$$I_x = I * D_x \text{ and } I_y = I * D_y.$$

Two common choices for D_x and D_y are the *Prewitt filters*

$$D_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \text{ and } D_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix},$$

and *Sobel filters*

$$D_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \text{ and } D_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

These derivative filters are easy to implement using the standard convolution available in the `scipy.ndimage.filters` module. For example:

```
from PIL import Image
from numpy import *
from scipy.ndimage import filters

im = array(Image.open('empire.jpg').convert('L'))
```

```
# Sobel derivative filters
imx = zeros(im.shape)
filters.sobel(im,1,imx)

imy = zeros(im.shape)
filters.sobel(im,0,imy)

magnitude = sqrt(imx**2+imy**2)
```

This computes x and y derivatives and gradient magnitude using the *Sobel filter*. The second argument selects the x or y derivative, and the third stores the output. Figure 1-10 shows an image with derivatives computed using the Sobel filter. In the two derivative images, positive derivatives are shown with bright pixels and negative derivatives are dark. Gray areas have values close to zero.

Using this approach has the drawback that derivatives are taken on the scale determined by the image resolution. To be more robust to image noise and to compute derivatives at any scale, *Gaussian derivative filters* can be used:

$$I_x = I * G_{\sigma_x} \text{ and } I_y = I * G_{\sigma_y},$$

where G_{σ_x} and G_{σ_y} are the x and y derivatives of G_{σ} , a Gaussian function with standard deviation σ .

The `filters.gaussian_filter()` function we used for blurring earlier can also take extra arguments to compute Gaussian derivatives instead. To try this on an image, simply do:

```
sigma = 5 # standard deviation

imx = zeros(im.shape)
filters.gaussian_filter(im, (sigma,sigma), (0,1), imx)

imy = zeros(im.shape)
filters.gaussian_filter(im, (sigma,sigma), (1,0), imy)
```

The third argument specifies which order of derivatives to use in each direction using the standard deviation determined by the second argument. See the documentation

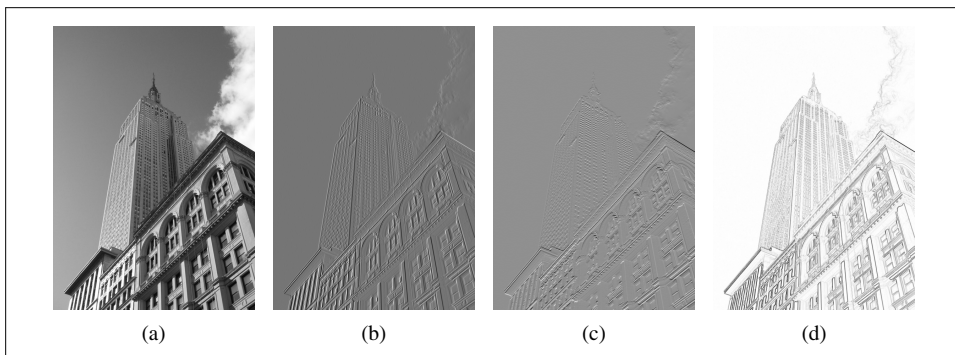


Figure 1-10. An example of computing image derivatives using Sobel derivative filters: (a) original image in grayscale; (b) x-derivative; (c) y-derivative; (d) gradient magnitude.

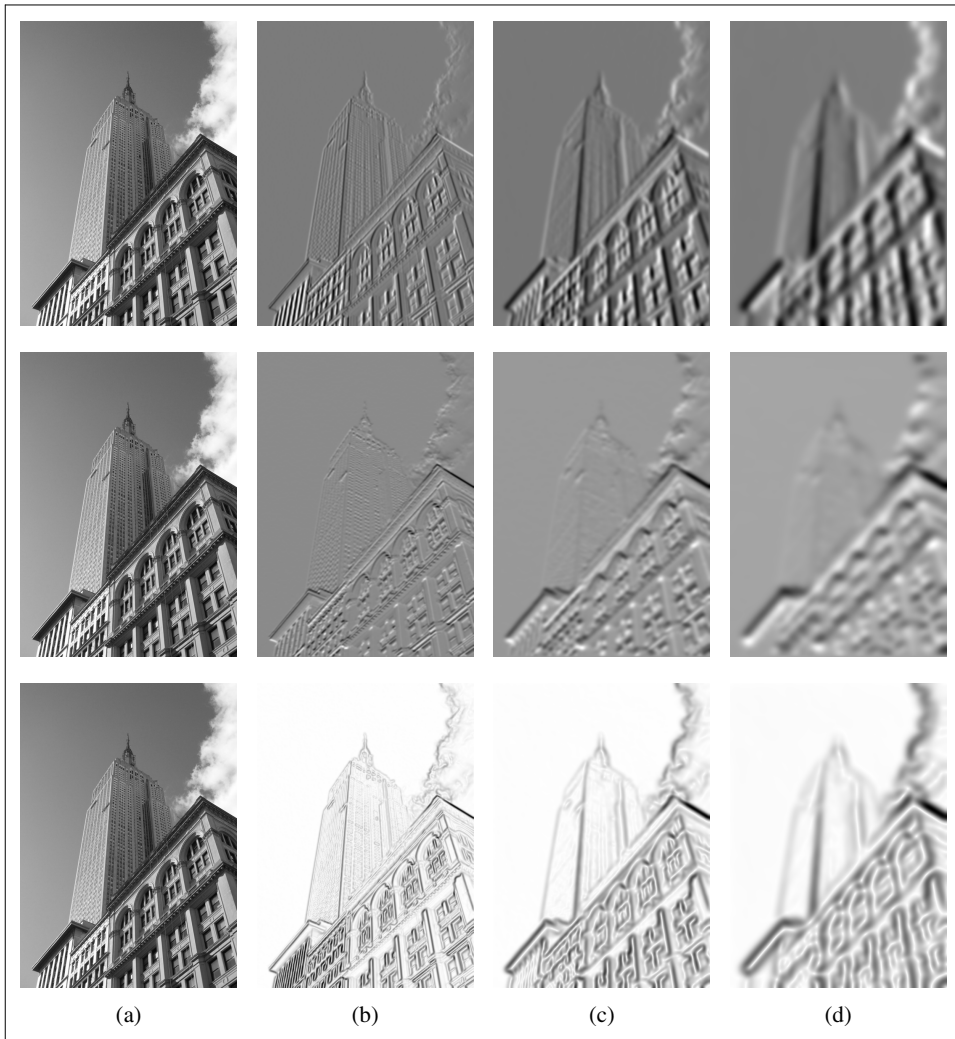


Figure 1-11. An example of computing image derivatives using Gaussian derivatives: *x*-derivative (top), *y*-derivative (middle), and gradient magnitude (bottom); (a) original image in grayscale, (b) Gaussian derivative filter with $\sigma = 2$, (c) with $\sigma = 5$, (d) with $\sigma = 10$.

for the details. Figure 1-11 shows the derivatives and gradient magnitude for different scales. Compare this to the blurring at the same scales in Figure 1-9.

Morphology—Counting Objects

Morphology (or *mathematical morphology*) is a framework and a collection of image processing methods for measuring and analyzing basic shapes. Morphology is usually applied to binary images but can be used with grayscale also. A *binary image* is an

image in which each pixel takes only two values, usually 0 and 1. Binary images are often the result of thresholding an image, for example with the intention of counting objects or measuring their size. A good summary of morphology and how it works is in http://en.wikipedia.org/wiki/Mathematical_morphology.

Morphological operations are included in the `scipy.ndimage` module `morphology`. Counting and measurement functions for binary images are in the `scipy.ndimage` module `measurements`. Let's look at a simple example of how to use them.

Consider the binary image in Figure 1-12a.³ Counting the objects in that image can be done using:

```
from scipy.ndimage import measurements,morphology

# load image and threshold to make sure it is binary
im = array(Image.open('houses.png').convert('L'))
im = 1*(im<128)

labels, nbr_objects = measurements.label(im)
print "Number of objects:", nbr_objects
```

This loads the image and makes sure it is binary by thresholding. Multiplying by 1 converts the boolean array to a binary one. Then the function `label()` finds the individual objects and assigns integer labels to pixels according to which object they belong to. Figure 1-12b shows the `labels` array. The graylevel values indicate object index. As you can see, there are small connections between some of the objects. Using an operation called binary opening, we can remove them:

```
# morphology - opening to separate objects better
im_open = morphology.binary_opening(im,ones((9,5)),iterations=2)

labels_open, nbr_objects_open = measurements.label(im_open)
print "Number of objects:", nbr_objects_open
```

The second argument of `binary_opening()` specifies the *structuring element*, an array that indicates what neighbors to use when centered around a pixel. In this case, we used 9 pixels (4 above, the pixel itself, and 4 below) in the y direction and 5 in the x direction. You can specify any array as structuring element; the non-zero elements will determine the neighbors. The parameter *iterations* determines how many times to apply the operation. Try this and see how the number of objects changes. The image after opening and the corresponding label image are shown in Figure 1-12c–d. As you might expect, there is a function named `binary_closing()` that does the reverse. We leave that and the other functions in `morphology` and `measurements` to the exercises. You can learn more about them from the `scipy.ndimage` documentation <http://docs.scipy.org/doc/scipy/reference/ndimage.html>.

³ This image is actually the result of image “segmentation.” Take a look at Section 93 if you want to see how this image was created.

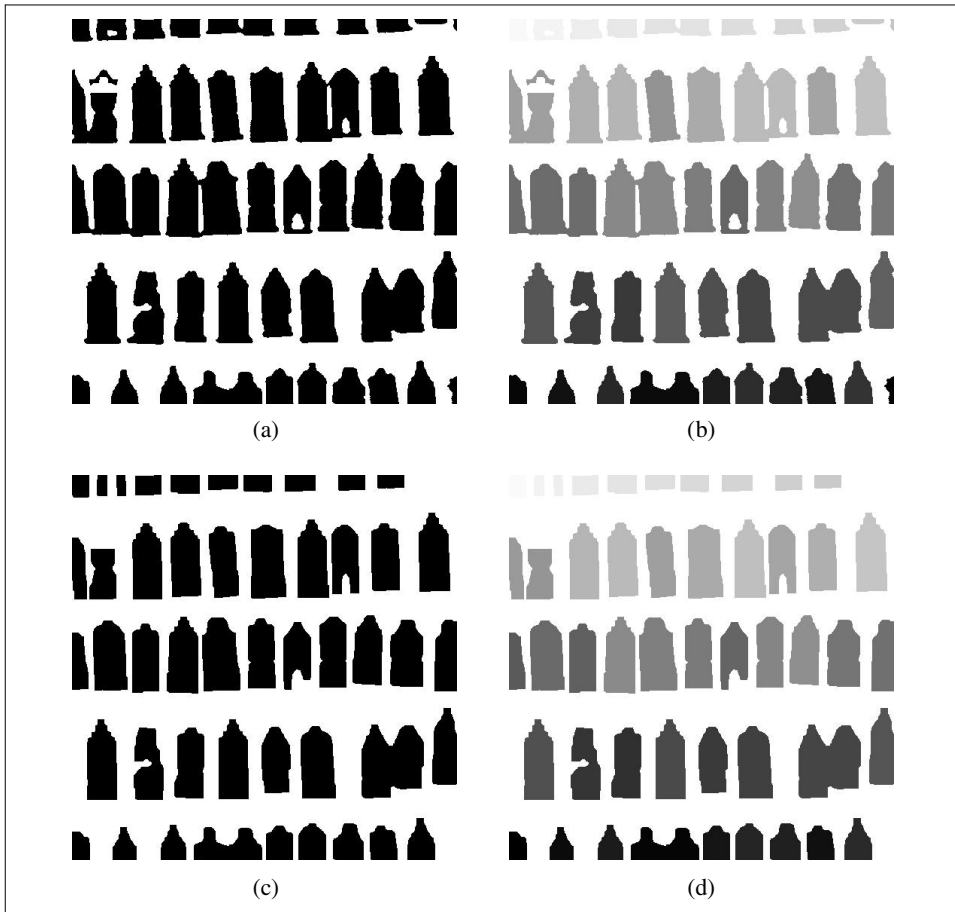


Figure 1-12. An example of morphology. Binary opening to separate objects followed by counting them: (a) original binary image; (b) label image corresponding to the original, grayvalues indicate object index; (c) binary image after opening; (d) label image corresponding to the opened image.

Useful SciPy Modules

SciPy comes with some useful modules for input and output. Two of them are `io` and `misc`.

Reading and writing `.mat` files

If you have some data, or find some interesting data set online, stored in Matlab's `.mat` file format, it is possible to read this using the `scipy.io` module. This is how to do it:

```
data = scipy.io.loadmat('test.mat')
```

The object `data` now contains a dictionary with keys corresponding to the variable names saved in the original `.mat` file. The variables are in array format. Saving to `.mat`

files is equally simple. Just create a dictionary with all variables you want to save and use `savemat()`:

```
data = {}
data['x'] = x
scipy.io.savemat('test.mat',data)
```

This saves the array `x` so that it has the name “x” when read into Matlab. More information on `scipy.io` can be found in the online documentation, <http://docs.scipy.org/doc/scipy/reference/io.html>.

Saving arrays as images

Since we are manipulating images and doing computations using array objects, it is useful to be able to save them directly as image files.⁴ Many images in this book are created just like this.

The `imsave()` function is available through the `scipy.misc` module. To save an array `im` to file just do the following:

```
from scipy.misc import imsave
imsave('test.jpg',im)
```

The `scipy.misc` module also contains the famous “Lena” test image:

```
lena = scipy.misc.lena()
```

This will give you a 512×512 grayscale array version of the image.

1.5 Advanced Example: Image De-Noising

We conclude this chapter with a very useful example, de-noising of images. Image *de-noising* is the process of removing image noise while at the same time trying to preserve details and structures. We will use the *Rudin-Osher-Fatemi de-noising model* (ROF) originally introduced in [28]. Removing noise from images is important for many applications, from making your holiday photos look better to improving the quality of satellite images. The ROF model has the interesting property that it finds a smoother version of the image while preserving edges and structures.

The underlying mathematics of the ROF model and the solution techniques are quite advanced and outside the scope of this book. We’ll give a brief, simplified introduction before showing how to implement a ROF solver based on an algorithm by Chambolle [5].

The *total variation* (TV) of a (grayscale) image I is defined as the sum of the gradient norm. In a continuous representation, this is

$$J(I) = \int |\nabla I| d\mathbf{x}. \quad (1.1)$$

⁴ All PyLab figures can be saved in a multitude of image formats by clicking the “save” button in the figure window.

In a discrete setting, the total variation becomes

$$J(I) = \sum_{\mathbf{x}} |\nabla I|,$$

where the sum is taken over all image coordinates $\mathbf{x} = [x, y]$.

In the Chambolle version of ROF, the goal is to find a de-noised image U that minimizes

$$\min_U ||I - U||^2 + 2\lambda J(U),$$

where the norm $||I - U||$ measures the difference between U and the original image I . What this means is, in essence, that the model looks for images that are “flat” but allows “jumps” at edges between regions.

Following the recipe in the paper, here’s the code:

```
from numpy import *

def denoise(im,U_init,tolerance=0.1,tau=0.125,tv_weight=100):
    """ An implementation of the Rudin-Osher-Fatemi (ROF) denoising model
        using the numerical procedure presented in eq (11) A. Chambolle (2005).

        Input: noisy input image (grayscale), initial guess for U, weight of
        the TV-regularizing term, steplength, tolerance for stop criterion.

        Output: denoised and detextured image, texture residual. """

    m,n = im.shape # size of noisy image

    # initialize
    U = U_init
    Px = im # x-component to the dual field
    Py = im # y-component of the dual field
    error = 1

    while (error > tolerance):
        Uold = U

        # gradient of primal variable
        GradUx = roll(U,-1,axis=1)-U # x-component of U's gradient
        GradUy = roll(U,-1,axis=0)-U # y-component of U's gradient

        # update the dual variable
        PxNew = Px + (tau/tv_weight)*GradUx
        PyNew = Py + (tau/tv_weight)*GradUy
        NormNew = maximum(1,sqrt(PxNew**2+PyNew**2))

        Px = PxNew/NormNew # update of x-component (dual)
        Py = PyNew/NormNew # update of y-component (dual)

        # update the primal variable
        RxPx = roll(Px,1,axis=1) # right x-translation of x-component
        RyPy = roll(Py,1,axis=0) # right y-translation of y-component

        DivP = (Px-RxPx)+(Py-RyPy) # divergence of the dual field.
```

```

U = im + tv_weight*DivP # update of the primal variable

# update of error
error = linalg.norm(U-Uold)/sqrt(n*m);

return U,im-U # denoised image and texture residual

```

In this example, we used the function `roll()`, which, as the name suggests, “rolls” the values of an array cyclically around an axis. This is very convenient for computing neighbor differences, in this case for derivatives. We also used `linalg.norm()`, which measures the difference between two arrays (in this case, the image matrices U and $Uold$). Save the function `denoise()` in a file `rof.py`.

Let’s start with a synthetic example of a noisy image:

```

from numpy import *
from numpy import random
from scipy.ndimage import filters
import rof

# create synthetic image with noise
im = zeros((500,500))
im[100:400,100:400] = 128
im[200:300,200:300] = 255
im = im + 30*random.standard_normal((500,500))

U,T = rof.denoise(im,im)
G = filters.gaussian_filter(im,10)

# save the result
from scipy.misc import imsave
imsave('synth_rof.pdf',U)
imsave('synth_gaussian.pdf',G)

```

The resulting images are shown in Figure 1-13 together with the original. As you can see, the ROF version preserves the edges nicely.

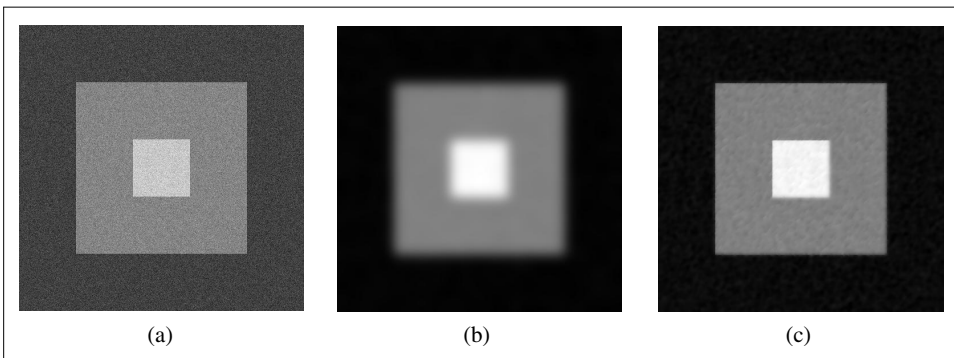


Figure 1-13. An example of ROF de-noising of a synthetic example: (a) original noisy image; (b) image after Gaussian blurring ($\sigma = 10$); (c) image after ROF de-noising.

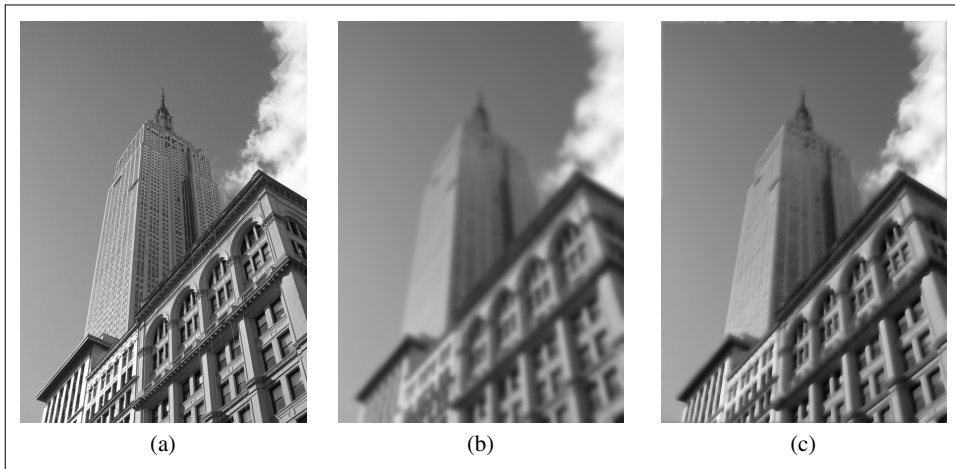


Figure 1-14. An example of ROF de-noising of a grayscale image: (a) original image; (b) image after Gaussian blurring ($\sigma = 5$); (c) image after ROF de-noising.

Now, let's see what happens with a real image:

```
from PIL import Image
from pylab import *
import rof

im = array(Image.open('empire.jpg').convert('L'))
U,T = rof.denoise(im,im)

figure()
gray()
imshow(U)
axis('equal')
axis('off')
show()
```

The result should look something like Figure 1-14c, which also shows a blurred version of the same image for comparison. As you can see, ROF de-noising preserves edges and image structures while at the same time blurring out the “noise.”

Exercises

1. Take an image and apply Gaussian blur like in Figure 1-9. Plot the image contours for increasing values of σ . What happens? Can you explain why?
2. Implement an *unsharp masking* operation (http://en.wikipedia.org/wiki/Unsharp_masking) by blurring an image and then subtracting the blurred version from the original. This gives a sharpening effect to the image. Try this on both color and grayscale images.

3. An alternative image normalization to histogram equalization is a *quotient image*. A quotient image is obtained by dividing the image with a blurred version $I/(I * G_\sigma)$. Implement this and try it on some sample images.
4. Write a function that finds the outline of simple objects in images (for example, a square against white background) using image gradients.
5. Use gradient direction and magnitude to detect lines in an image. Estimate the extent of the lines and their parameters. Plot the lines overlaid on the image.
6. Apply the `label()` function to a thresholded image of your choice. Use histograms and the resulting label image to plot the distribution of object sizes in the image.
7. Experiment with successive morphological operations on a thresholded image of your choice. When you have found some settings that produce good results, try the function `center_of_mass` in `morphology` to find the center coordinates of each object and plot them in the image.

Conventions for the Code Examples

From Chapter 2 and onward, we assume `PIL`, `NumPy`, and `Matplotlib` are included at the top of every file you create and in every code example as:

```
from PIL import Image
from numpy import *
from pylab import *
```

This makes the example code cleaner and the presentation easier to follow. In the cases when we use `SciPy` modules, we will explicitly declare that in the examples.

Purists will object to this type of blanket imports and insist on something like

```
import numpy as np
import matplotlib.pyplot as plt
```

so that namespaces can be kept (to know where each function comes from) and only import the `pyplot` part of `Matplotlib`, since the `NumPy` parts imported with `PyLab` are not needed. Purists and experienced programmers know the difference and can choose whichever option they prefer. In the interest of making the content and examples in this book easily accessible to readers, I have chosen not to do this.

Caveat emptor.

Local Image Descriptors

This chapter is about finding corresponding points and regions between images. Two different types of local descriptors are introduced with methods for matching these between images. These local features will be used in many different contexts throughout this book and are an important building block in many applications, such as creating panoramas, augmented reality, and computing 3D reconstructions.

2.1 Harris Corner Detector

The *Harris corner detection* algorithm (or sometimes the Harris & Stephens corner detector) is one of the simplest corner indicators available. The general idea is to locate interest points where the surrounding neighborhood shows edges in more than one direction; these are then image corners.

We define a matrix $\mathbf{M}_I = \mathbf{M}_I(\mathbf{x})$, on the points $\mathbf{x}$ in the image domain, as the positive semi-definite, symmetric matrix

$$\mathbf{M}_I = \nabla I \nabla I^T = \begin{bmatrix} I_x \\ I_y \end{bmatrix} \begin{bmatrix} I_x & I_y \end{bmatrix} = \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}, \quad (2.1)$$

whereas before ∇I is the image gradient containing the derivatives I_x and I_y (we defined the derivatives and the gradient on page 18). Because of this construction, $\mathbf{M}_I$ has rank one with eigenvalues $\lambda_1 = |\nabla I|^2$ and $\lambda_2 = 0$. We now have one matrix for each pixel in the image.

Let W be a weight matrix (typically a Gaussian filter G_σ). The component-wise convolution

$$\overline{\mathbf{M}}_I = W * \mathbf{M}_I \quad (2.2)$$

gives a local averaging of $\mathbf{M}_I$ over the neighboring pixels. The resulting matrix $\overline{\mathbf{M}}_I$ is sometimes called a *Harris matrix*. The width of W determines a region of interest around $\mathbf{x}$. The idea of averaging the matrix $\mathbf{M}_I$ over a region like this is that the eigenvalues will change depending on the local image properties. If the gradients vary

in the region, the second eigenvalue of $\overline{\mathbf{M}}_I$ will no longer be zero. If the gradients are the same, the eigenvalues will be the same as for $\mathbf{M}_I$.

Depending on the values of ∇I in the region, there are three cases for the eigenvalues of the Harris matrix, $\overline{\mathbf{M}}_I$:

- If λ_1 and λ_2 are both large positive values, then there is a corner at $\mathbf{x}$.
- If λ_1 is large and $\lambda_2 \approx 0$, then there is an edge and the averaging of $\mathbf{M}_I$ over the region doesn't change the eigenvalues that much.
- If $\lambda_1 \approx \lambda_2 \approx 0$, then there is nothing.

To distinguish the important case from the others without actually having to compute the eigenvalues, Harris and Stephens [12] introduced an indicator function

$$\det(\overline{\mathbf{M}}_I) - \kappa \text{trace}(\overline{\mathbf{M}}_I)^2.$$

To get rid of the weighting constant κ , it is often easier to use the quotient

$$\frac{\det(\overline{\mathbf{M}}_I)}{\text{trace}(\overline{\mathbf{M}}_I)^2}$$

as an indicator.

Let's see what this looks like in code. For this, we need the `scipy.ndimage.filters` module for computing derivatives using Gaussian derivative filters as described on page 18. The reason is again that we would like to suppress noise sensitivity in the corner detection process.

First, add the corner response function to a file *harris.py*, which will make use of the Gaussian derivatives. Again, the parameter σ defines the scale of the Gaussian filters used. You can also modify this function to take different scales in the x and y directions, as well as a different scale for the averaging, to compute the Harris matrix.

```
from scipy.ndimage import filters

def compute_harris_response(im,sigma=3):
    """ Compute the Harris corner detector response function
        for each pixel in a graylevel image. """

    # derivatives
    imx = zeros(im.shape)
    filters.gaussian_filter(im, (sigma,sigma), (0,1), imx)
    imy = zeros(im.shape)
    filters.gaussian_filter(im, (sigma,sigma), (1,0), imy)

    # compute components of the Harris matrix
    Wxx = filters.gaussian_filter(imx*imx,sigma)
    Wxy = filters.gaussian_filter(imx*imy,sigma)
    Wyy = filters.gaussian_filter(imy*imy,sigma)

    # determinant and trace
    Wdet = Wxx*Wyy - Wxy**2
    Wtr = Wxx + Wyy

    return Wdet / Wtr
```


This gives an image with each pixel containing the value of the Harris response function. Now, it is just a matter of picking out the information needed from this image. Taking all points with values above a threshold, with the additional constraint that corners must be separated with a minimum distance, is an approach that often gives good results. To do this, take all candidate pixels, sort them in descending order of corner response values, and mark off regions too close to positions already marked as corners. Add the following function to *harris.py*:

```
def get_harris_points(harrisim,min_dist=10,threshold=0.1):
    """ Return corners from a Harris response image
        min_dist is the minimum number of pixels separating
        corners and image boundary. """

    # find top corner candidates above a threshold
    corner_threshold = harrisim.max() * threshold
    harrisim_t = (harrisim > corner_threshold) * 1

    # get coordinates of candidates
    coords = array(harrisim_t.nonzero()).T

    # ...and their values
    candidate_values = [harrisim[c[0],c[1]] for c in coords]

    # sort candidates
    index = argsort(candidate_values)

    # store allowed point locations in array
    allowed_locations = zeros(harrisim.shape)
    allowed_locations[min_dist:-min_dist,min_dist:-min_dist] = 1

    # select the best points taking min_distance into account
    filtered_coords = []
    for i in index:
        if allowed_locations[coords[i,0],coords[i,1]] == 1:
            filtered_coords.append(coords[i])
            allowed_locations[(coords[i,0]-min_dist):(coords[i,0]+min_dist),
                             (coords[i,1]-min_dist):(coords[i,1]+min_dist)] = 0

    return filtered_coords
```

Now you have all you need to detect corner points in images. To show the corner points in the image, you can add a plotting function to *harris.py* using *Matplotlib* as follows:

```
def plot_harris_points(image,filtered_coords):
    """ Plots corners found in image. """

    figure()
    gray()
    imshow(image)
    plot([p[1] for p in filtered_coords],[p[0] for p in filtered_coords],'*')
    axis('off')
    show()
```

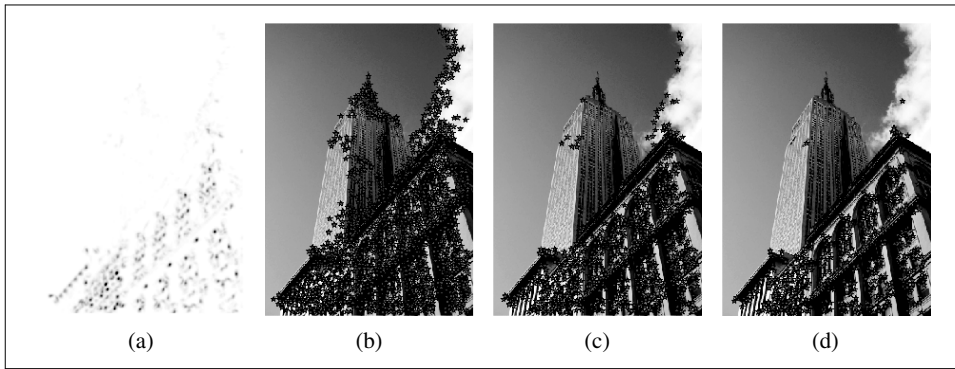


Figure 2-1. An example of corner detection with the Harris corner detector: (a) the Harris response function; (b–d) corners detected with threshold 0.01, 0.05, and 0.1, respectively.

Try running the following commands:

```
im = array(Image.open('empire.jpg').convert('L'))
harrisim = harris.compute_harris_response(im)
filtered_coords = harris.get_harris_points(harrisim,6)
harris.plot_harris_points(im, filtered_coords)
```

The image is opened and converted to grayscale. Then, the response function is computed and points selected based on the response values. Finally, the points are plotted overlaid on the original image. This should give you a plot like the images in Figure 2-1.

For an overview of different approaches to corner detection, including improvements on the Harris detector and further developments, see for example http://en.wikipedia.org/wiki/Corner_detection.

Finding Corresponding Points Between Images

The Harris corner detector gives interest points in images but does not contain an inherent way of comparing these interest points across images to find matching corners. What we need is to add a descriptor to each point and a way to compare such descriptors.

An *interest point descriptor* is a vector assigned to an interest point that describes the image appearance around the point. The better the descriptor, the better your correspondences will be. By *point correspondence* or *corresponding points*, we mean points in different images that refer to the same object or scene point.

Harris corner points are usually combined with a descriptor consisting of the graylevel values in a neighboring image patch together with normalized cross-correlation for comparison. An *image patch* is almost always a rectangular portion of the image centered around the point in question.

In general, *correlation* between two (equally sized) image patches $I_1(\mathbf{x})$ and $I_2(\mathbf{x})$ is defined as

$$c(I_1, I_2) = \sum_{\mathbf{x}} f(I_1(\mathbf{x}), I_2(\mathbf{x})),$$

where the function f varies depending on the correlation method. The sum is taken over all positions $\mathbf{x}$ in the image patches. For *cross-correlation*, the function is $f(I_1, I_2) = I_1 \cdot I_2$, and then $c(I_1, I_2) = I_1 \cdot I_2$, with $\cdot$ denoting the scalar product (of the row- or column-stacked patches). The larger the value of $c(I_1, I_2)$, the more similar the patches I_1 and I_2 are.¹

Normalized cross-correlation is a variant of cross-correlation defined as

$$ncc(I_1, I_2) = \frac{1}{n-1} \sum_{\mathbf{x}} \frac{(I_1(\mathbf{x}) - \mu_1)}{\sigma_1} \cdot \frac{(I_2(\mathbf{x}) - \mu_2)}{\sigma_2}, \quad (2.3)$$

where n is the number of pixels in a patch, μ_1 and μ_2 are the mean intensities, and σ_1 and σ_2 are the standard deviations in each patch, respectively. By subtracting the mean and scaling with the standard deviation, the method becomes robust to changes in image brightness.

To extract image patches and compare them using normalized cross-correlation, you need two more functions in *harris.py*. Add these:

```
def get_descriptors(image, filtered_coords, wid=5):
    """ For each point return, pixel values around the point
        using a neighbourhood of width 2*wid+1. (Assume points are
        extracted with min_distance > wid). """

    desc = []
    for coords in filtered_coords:
        patch = image[coords[0]-wid:coords[0]+wid+1,
                      coords[1]-wid:coords[1]+wid+1].flatten()
        desc.append(patch)

    return desc

def match(desc1, desc2, threshold=0.5):
    """ For each corner point descriptor in the first image,
        select its match to second image using
        normalized cross-correlation. """

    n = len(desc1[0])

    # pair-wise distances
    d = -ones((len(desc1), len(desc2)))
    for i in range(len(desc1)):
        for j in range(len(desc2)):
            d1 = (desc1[i] - mean(desc1[i])) / std(desc1[i])
            d2 = (desc2[j] - mean(desc2[j])) / std(desc2[j])
            ncc_value = sum(d1 * d2) / (n-1)
            if ncc_value > threshold:
                d[i, j] = ncc_value

    ndx = argsort(-d)
    matchscores = ndx[:, 0]

    return matchscores
```

¹ Another popular function is $f(I_1, I_2) = (I_1 - I_2)^2$, which gives *sum of squared differences* (SSD).

The first function takes a square grayscale patch of odd side length centered around the point, flattens it, and adds to a list of descriptors. The second function matches each descriptor to its best candidate in the other image using normalized cross-correlation. Note that the distances are negated before sorting, since a high value means better match. To further stabilize the matches, we can match from the second image to the first and filter out the matches that are not the best both ways. The following function does just that:

```
def match_twosided(desc1,desc2,threshold=0.5):
    """ Two-sided symmetric version of match(). """

    matches_12 = match(desc1,desc2,threshold)
    matches_21 = match(desc2,desc1,threshold)

    ndx_12 = where(matches_12 >= 0)[0]

    # remove matches that are not symmetric
    for n in ndx_12:
        if matches_21[matches_12[n]] != n:
            matches_12[n] = -1

    return matches_12
```

The matches can be visualized by showing the images side-by-side and connecting matched points with lines using the following code. Add these two functions to *harris.py*:

```
def appendimages(im1,im2):
    """ Return a new image that appends the two images side-by-side. """

    # select the image with the fewest rows and fill in enough empty rows
    rows1 = im1.shape[0]
    rows2 = im2.shape[0]

    if rows1 < rows2:
        im1 = concatenate((im1,zeros((rows2-rows1,im1.shape[1]))),axis=0)
    elif rows1 > rows2:
        im2 = concatenate((im2,zeros((rows1-rows2,im2.shape[1]))),axis=0)
    # if none of these cases they are equal, no filling needed.

    return concatenate((im1,im2), axis=1)

def plot_matches(im1,im2,locs1,locs2,matchscores,show_below=True):
    """ Show a figure with lines joining the accepted matches
        input: im1,im2 (images as arrays), locs1,locs2 (feature locations),
               matchscores (as output from 'match()'),
               show_below (if images should be shown below matches). """

    im3 = appendimages(im1,im2)
    if show_below:
        im3 = vstack((im3,im3))
```



Figure 2-2. Example of matches resulting from applying normalized cross-correlation to patches around Harris corner points.

```
imshow(im3)

cols1 = im1.shape[1]
for i,m in enumerate(matchescores):
    if m>0:
        plot([locs1[i][1],locs2[m][1]+cols1],[locs1[i][0],locs2[m][0]],'c')
axis('off')
```

Figure 2-2 shows an example of finding such corresponding points using normalized cross-correlation (in this case, with 11×11 pixels in a patch) using the following commands:

```

wid = 5
harrisim = harris.compute_harris_response(im1,5)
filtered_coords1 = harris.get_harris_points(harrisim,wid+1)
d1 = harris.get_descriptors(im1,filtered_coords1,wid)

harrisim = harris.compute_harris_response(im2,5)
filtered_coords2 = harris.get_harris_points(harrisim,wid+1)
d2 = harris.get_descriptors(im2,filtered_coords2,wid)

print 'starting matching'
matches = harris.match_twosided(d1,d2)

figure()
gray()
harris.plot_matches(im1,im2,filtered_coords1,filtered_coords2,matches)
show()

```

If you only want to plot a subset of the matches to make the visualization clearer, substitute *matches* with, for example, *matches[:100]* or a random set of indices.

As you can see in Figure 2-2, there are quite a lot of incorrect matches. This is because cross-correlation on image patches is not as descriptive as more modern approaches. As a consequence, it is important to use robust methods for handling these correspondences in an application. Another problem is that these descriptors are not invariant to scale or rotation, and the choice of patch sizes affects the results.

In recent years, there has been a lot of development in improving feature point detection and description. Let's take a look at one of the best algorithms in the next section.

2.2 SIFT—Scale-Invariant Feature Transform

One of the most successful local image descriptors in the last decade is the *Scale-Invariant Feature Transform (SIFT)*, introduced by David Lowe in [17]. SIFT was later refined and described in detail in the paper [18] and has stood the test of time. SIFT includes both an interest point detector and a descriptor. The descriptor is very robust and is largely the reason behind the success and popularity of SIFT. Since its introduction, many alternatives have been proposed with essentially the same type of descriptor. The descriptor is nowadays often combined with many different interest point detectors (and region detectors for that matter) and sometimes even applied densely across the whole image. SIFT features are invariant to scale, rotation, and intensity and can be matched reliably across 3D viewpoint and noise. A brief overview is available online at http://en.wikipedia.org/wiki/Scale-invariant_feature_transform.

Interest Points

SIFT interest point locations are found using *difference-of-Gaussian* functions

$$D(\mathbf{x}, \sigma) = [G_{k\sigma}(\mathbf{x}) - G_{\sigma}(\mathbf{x})] * I(\mathbf{x}) = [G_{k\sigma} - G_{\sigma}] * I = I_{k\sigma} - I_{\sigma},$$

where G_{σ} is the Gaussian 2D kernel described on page 16, I_{σ} the G_{σ} -blurred grayscale image, and k a constant factor determining the separation in scale. Interest points are the maxima and minima of $D(\mathbf{x}, \sigma)$ across both image location and scale. These candidate